11.6 Red-Black Trees

Although AVL trees and $(2,4)$ trees have a number of nice properties, they also have some disadvantages. For instance, AVL trees may require many restructure operations (rotations) to be performed after a deletion, and $(2,4)$ trees may require many split or fusing operations to be performed after an insertion or removal. The data structure we discuss in this section, the red-black tree, does not have these drawbacks; it uses $O(1)$ structural changes after an update in order to stay balanced.

Formally, a ***red-black tree*** is a binary search tree (see Section 11.1) with nodes colored red and black in a way that satisfies the following properties:

***Root Property*:** The root is black.

***Red Property*:** The children of a red node (if any) are black.

Depth Property*:** All nodes with zero or one children have the same ***black depth, defined as the number of black ancestors. (Recall that a node is its own ancestor).

An example of a red-black tree is shown in Figure 11.30.

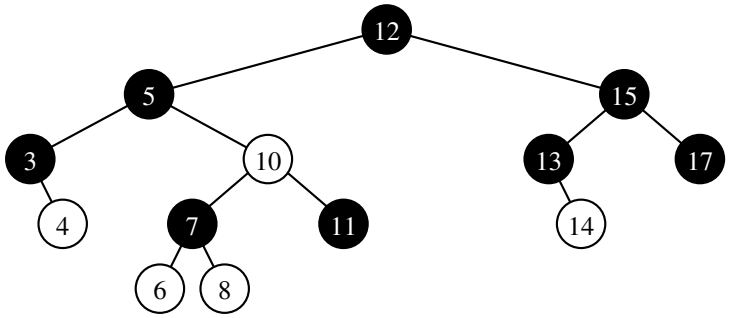


Figure 11.30: An example of a red-black tree, with "red" nodes drawn in white. The common black depth for this tree is 3.

We can make the red-black tree definition more intuitive by noting an interesting correspondence between red-black trees and $(2,4)$ trees (excluding their trivial external nodes). Namely, given a red-black tree, we can construct a corresponding $(2,4)$ tree by merging every red node w into its parent, storing the entry from w at its parent, and with the children of w becoming ordered children of the parent. For example, the red-black tree in Figure 11.30 corresponds to the $(2,4)$ tree from Figure 11.24, as illustrated in Figure 11.31. The depth property of the red-black tree corresponds to the depth property of the $(2,4)$ tree since exactly one black node of the red-black tree contributes to each node of the corresponding $(2,4)$ tree.

Conversely, we can transform any $(2,4)$ tree into a corresponding red-black tree by coloring each node w black and then performing the following transformations, as illustrated in Figure 11.32.

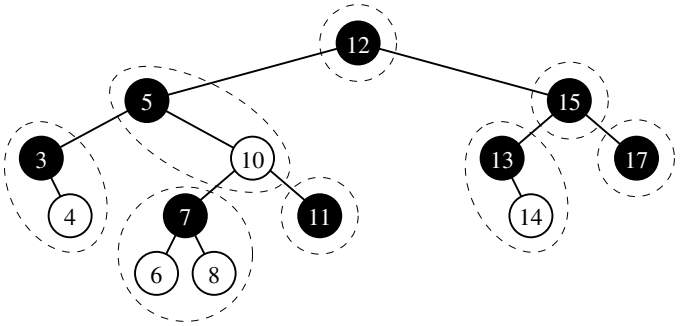


Figure 11.31: An illustration that the red-black tree of Figure 11.30 corresponds to the $(2,4)$ tree of Figure 11.24, based on the highlighted grouping of red nodes with their black parents.

- If w is a 2-node, then keep the (black) children of w as is.
- If w is a 3-node, then create a new red node y, give w's last two (black) children to y, and make the first child of w and y be the two children of w.
- If w is a 4-node, then create two new red nodes y and z, give w's first two (black) children to y, give w's last two (black) children to z, and make y and z be the two children of w.

Notice that a red node always has a black parent in this construction.

Proposition 11.9: *The height of a red-black tree storing n entries is $O(\log n)$.*

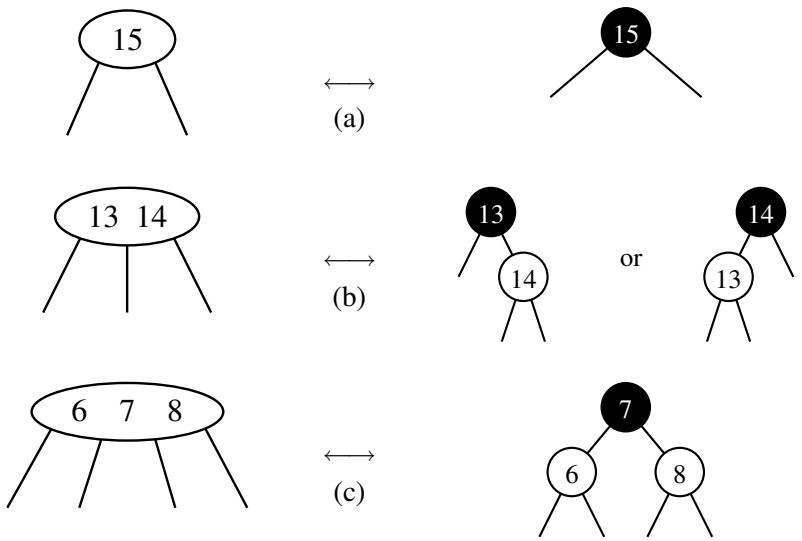


Figure 11.32: Correspondence between nodes of a $(2,4)$ tree and a red-black tree: (a) 2-node; (b) 3-node; (c) 4-node.

Justification: Let T be a red-black tree storing n entries, and let h be the height of T. We justify this proposition by establishing the following fact:

$$\log(n+1) - 1 \leq h \leq 2\log(n+1) - 2.$$

Let d be the common black depth of all nodes of T having zero or one children. Let T' be the $(2,4)$ tree associated with T, and let h' be the height of T' (excluding trivial leaves). Because of the correspondence between red-black trees and $(2,4)$ trees, we know that $h' = d$. Hence, by Proposition 11.8, $d = h' \leq \log(n+1) - 1$. By the red property, $h \leq 2d$. Thus, we obtain $h \leq 2\log(n+1) - 2$. The other inequality, $\log(n+1) - 1 \leq h$, follows from Proposition 8.8 and the fact that T has n nodes. ■

11.6.1 Red-Black Tree Operations

The algorithm for searching in a red-black tree T is the same as that for a standard binary search tree (Section 11.1). Thus, searching in a red-black tree takes time proportional to the height of the tree, which is $O(\log n)$ by Proposition 11.9.

The correspondence between $(2,4)$ trees and red-black trees provides important intuition that we will use in our discussion of how to perform updates in red-black trees; in fact, the update algorithms for red-black trees can seem mysteriously complex without this intuition. Split and fuse operations of a $(2,4)$ tree will be effectively mimicked by recoloring neighboring red-black tree nodes. A rotation within a red-black tree will be used to change orientations of a 3-node between the two forms shown in Figure 11.32(b).

Insertion

Now consider the insertion of a key-value pair (k,v) into a red-black tree T. The algorithm initially proceeds as in a standard binary search tree (Section 11.1.3). Namely, we search for k in T until we reach a null subtree, and we introduce a new leaf x at that position, storing the item. In the special case that x is the only node of T, and thus the root, we color it black. In all other cases, we color x red. This action corresponds to inserting (k,v) into a node of the $(2,4)$ tree T' with external children. The insertion preserves the root and depth properties of T, but it may violate the red property. Indeed, if x is not the root of T and the parent y of x is red, then we have a parent and a child (namely, y and x) that are both red. Note that by the root property, y cannot be the root of T, and by the red property (which was previously satisfied), the parent z of y must be black. Since x and its parent are red, but x's grandparent z is black, we call this violation of the red property a ***double red*** at node x. To remedy a double red, we consider two cases.

Case 1: ***The Sibling s of y Is Black (or None).*** (See Figure 11.33.) In this case, the double red denotes the fact that we have added the new node to a corresponding 3-node of the $(2,4)$ tree T', effectively creating a malformed 4-node. This formation has one red node (y) that is the parent of another red node (x), while we want it to have the two red nodes as siblings instead. To fix this problem, we perform a ***trinode restructuring*** of T. The trinode restructuring is done by the operation $\mathsf{restructure}(x)$, which consists of the following steps (see again Figure 11.33; this operation is also discussed in Section 11.2):

- Take node x, its parent y, and grandparent z, and temporarily relabel them as a, b, and c, in left-to-right order, so that a, b, and c will be visited in this order by an inorder tree traversal.
- Replace the grandparent z with the node labeled b, and make nodes a and c the children of b, keeping inorder relationships unchanged.

After performing the $\mathsf{restructure}(x)$ operation, we color b black and we color a and c red. Thus, the restructuring eliminates the double-red problem. Notice that the portion of any path through the restructured part of the tree is incident to exactly one black node, both before and after the trinode restructuring. Therefore, the black depth of the tree is unaffected.

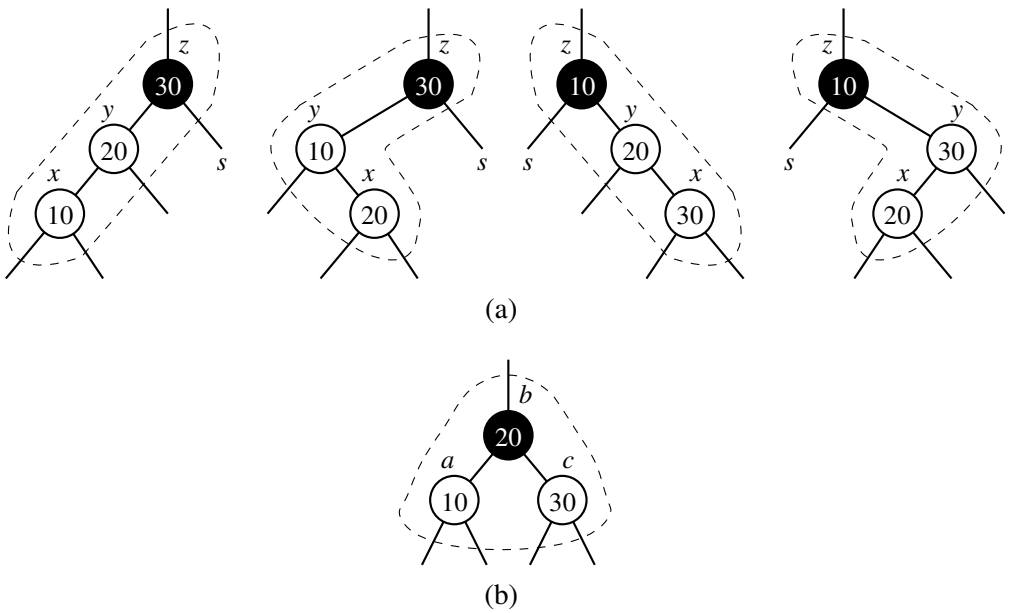


Figure 11.33: Restructuring a red-black tree to remedy a double red: (a) the four configurations for x, y, and z before restructuring; (b) after restructuring.

Case 2: ***The Sibling s of y Is Red.*** (See Figure 11.34.) In this case, the double red denotes an overflow in the corresponding $(2,4)$ tree T'. To fix the problem, we perform the equivalent of a split operation. Namely, we do a ***recoloring***: we color y and s black and their parent z red (unless z is the root, in which case, it remains black). Notice that unless z is the root, the portion of any path through the affected part of the tree is incident to exactly one black node, both before and after the recoloring. Therefore, the black depth of the tree is unaffected by the recoloring unless z is the root, in which case it is increased by one.

However, it is possible that the double-red problem reappears after such a recoloring, albeit higher up in the tree T, since z may have a red parent. If the double-red problem reappears at z, then we repeat the consideration of the two cases at z. Thus, a recoloring either eliminates the double-red problem at node x, or propagates it to the grandparent z of x. We continue going up T performing recolorings until we finally resolve the double-red problem (with either a final recoloring or a trinode restructuring). Thus, the number of recolorings caused by an insertion is no more than half the height of tree T, that is, $O(\log n)$ by Proposition 11.9.

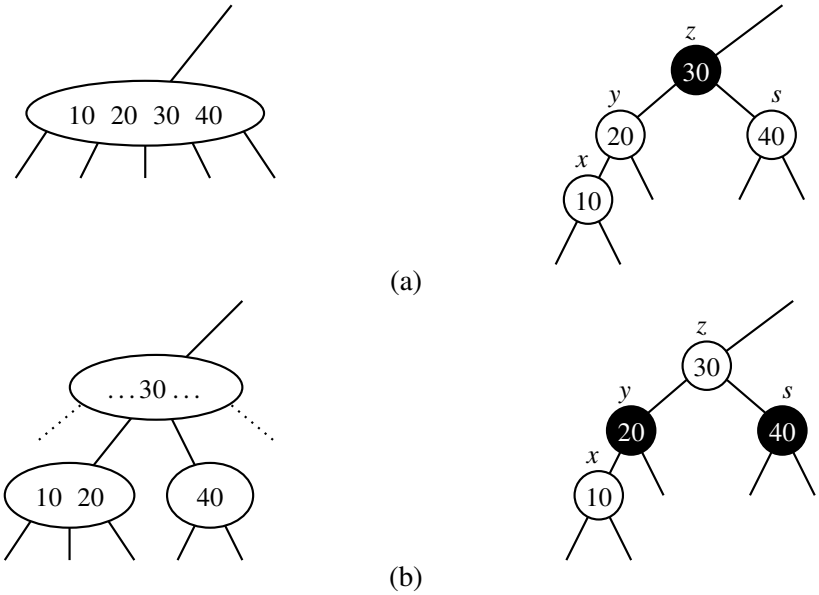


Figure 11.34: Recoloring to remedy the double-red problem: (a) before recoloring and the corresponding 5-node in the associated $(2,4)$ tree before the split; (b) after recoloring and the corresponding nodes in the associated $(2,4)$ tree after the split.

As further examples, Figures 11.35 and 11.36 show a sequence of insertion operations in a red-black tree.

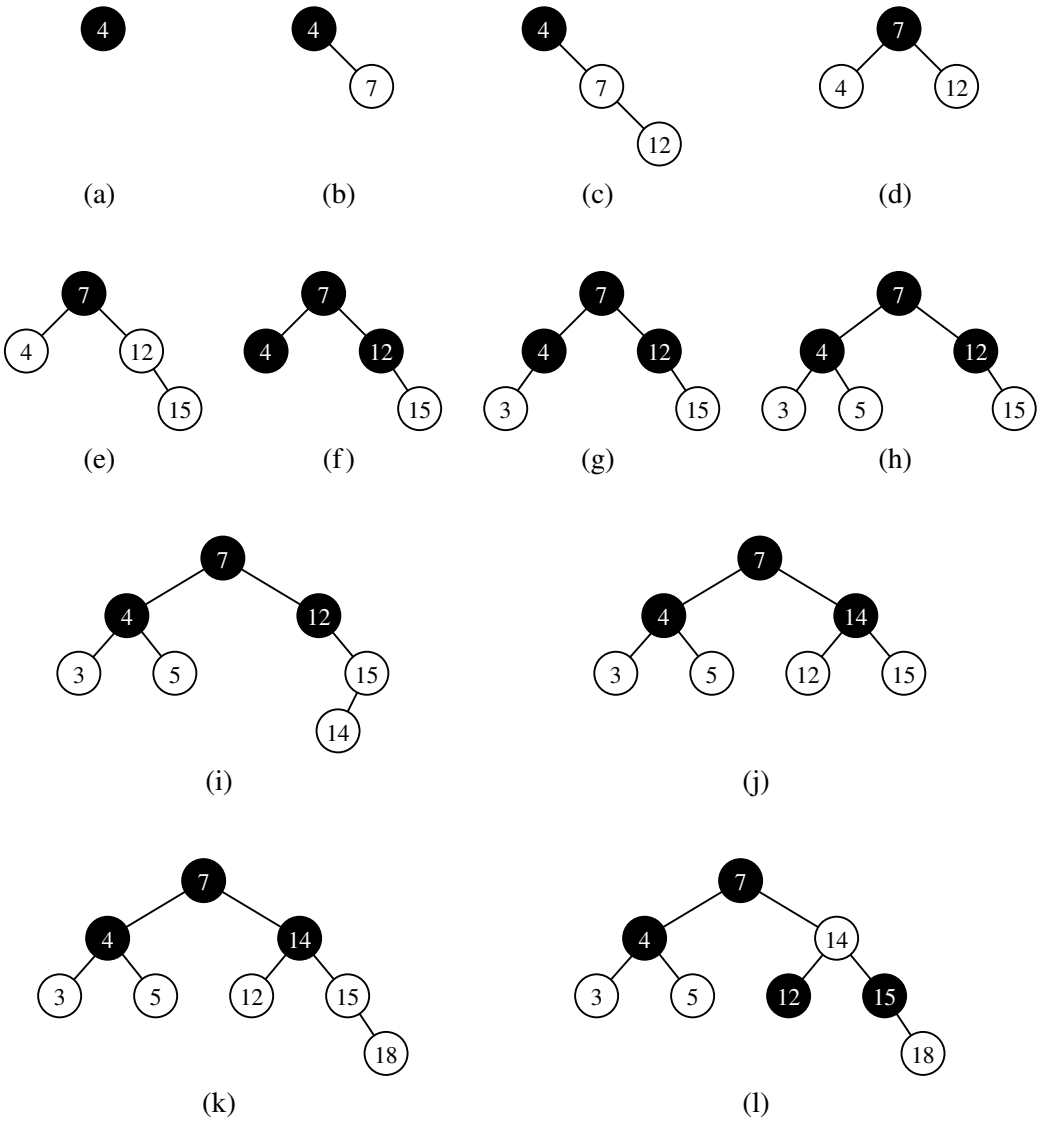


Figure 11.35: A sequence of insertions in a red-black tree: (a) initial tree; (b) insertion of 7; (c) insertion of 12, which causes a double red; (d) after restructuring; (e) insertion of 15, which causes a double red; (f) after recoloring (the root remains black); (g) insertion of 3; (h) insertion of 5; (i) insertion of 14, which causes a double red; (j) after restructuring; (k) insertion of 18, which causes a double red; (l) after recoloring. (Continues in Figure 11.36.)

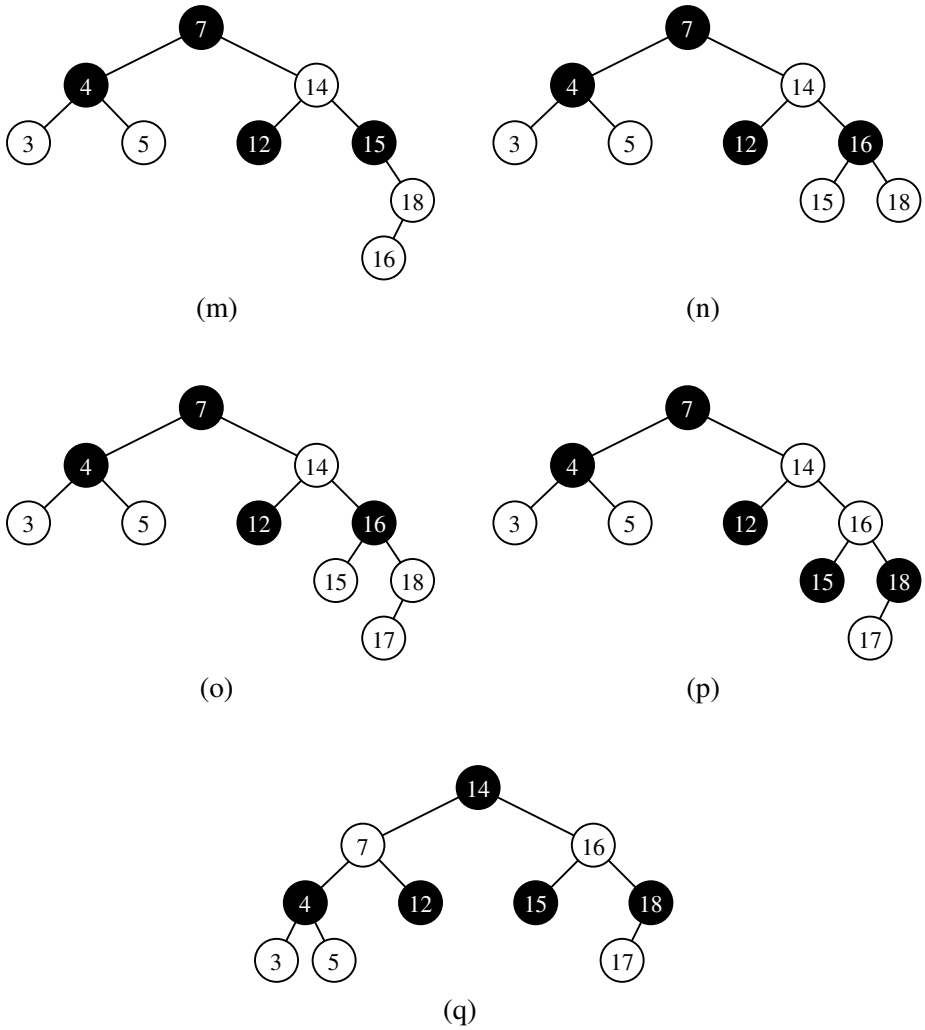


Figure 11.36: A sequence of insertions in a red-black tree: (m) insertion of 16, which causes a double red; (n) after restructuring; (o) insertion of 17, which causes a double red; (p) after recoloring there is again a double red, to be handled by a restructuring; (q) after restructuring. (Continued from Figure 11.35.)

Deletion

Deleting an item with key k from a red-black tree T initially proceeds as for a binary search tree (Section 11.1.3). Structurally, the process results in the removal a node that has at most one child (either that originally containing key k or its inorder predecessor) and the promotion of its remaining child (if any).

If the removed node was red, this structural change does not affect the black depths of any paths in the tree, nor introduce any red violations, and so the resulting tree remains a valid red-black tree. In the corresponding $(2,4)$ tree T', this case denotes the shrinking of a 3-node or 4-node. If the removed node was black, then it either had zero children or it had one child that was a red leaf (because the null subtree of the removed node has black height 0). In the latter case, the removed node represents the black part of a corresponding 3-node, and we restore the red-black properties by recoloring the promoted child to black.

The more complex case is when a (nonroot) black leaf is removed. In the corresponding $(2,4)$ tree, this denotes the removal of an item from a 2-node. Without rebalancing, such a change results in a deficit of one for the black depth along the path leading to the deleted item. By necessity, the removed node must have a sibling whose subtree has black height 1 (given that this was a valid red-black tree prior to the deletion of the black leaf).

To remedy this scenario, we consider a more general setting with a node z that is known to have two subtrees, T_{heavy} and T_{light}, such that the root of T_{light} (if any) is black and such that the black depth of T_{heavy} is exactly one more than that of T_{light}, as portrayed in Figure 11.37. In the case of a removed black leaf, z is the parent of that leaf and T_{light} is trivially the empty subtree that remains after the deletion. We describe the more general case of a deficit because our algorithm for rebalancing the tree will, in some cases, push the deficit higher in the tree (just as the resolution of a deletion in a $(2,4)$ tree sometimes cascades upward). We let y denote the root of T_{heavy}. (Such a node exists because T_{heavy} has black height at least one.)

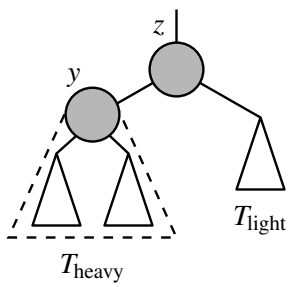


Figure 11.37: Portrayal of a deficit between the black heights of subtrees of node z. The gray color in illustrating y and z denotes the fact that these nodes may be colored either black or red.

We consider three possible cases to remedy a deficit.

Case 1: ***Node y Is Black and Has a Red Child x***. (See Figure 11.38.)

We perform a ***trinode restructuring***, as originally described in Section 11.2. The operation restructure(x) takes the node x, its parent y, and grandparent z, labels them temporarily left to right as a, b, and c, and replaces z with the node labeled b, making it the parent of the other two. We color a and c black, and give b the former color of z.

Notice that the path to T_{light} in the result includes one additional black node after the restructure, thereby resolving its deficit. In contrast, the number of black nodes on paths to any of the other three subtrees illustrated in Figure 11.38 remains unchanged.

Resolving this case corresponds to a transfer operation in the $(2,4)$ tree T' between the two children of the node with z. The fact that y has a red child assures us that it represents either a 3-node or a 4-node. In effect, the item previously stored at z is demoted to become a new 2-node to resolve the deficiency, while an item stored at y or its child is promoted to take the place of the item previously stored at z.

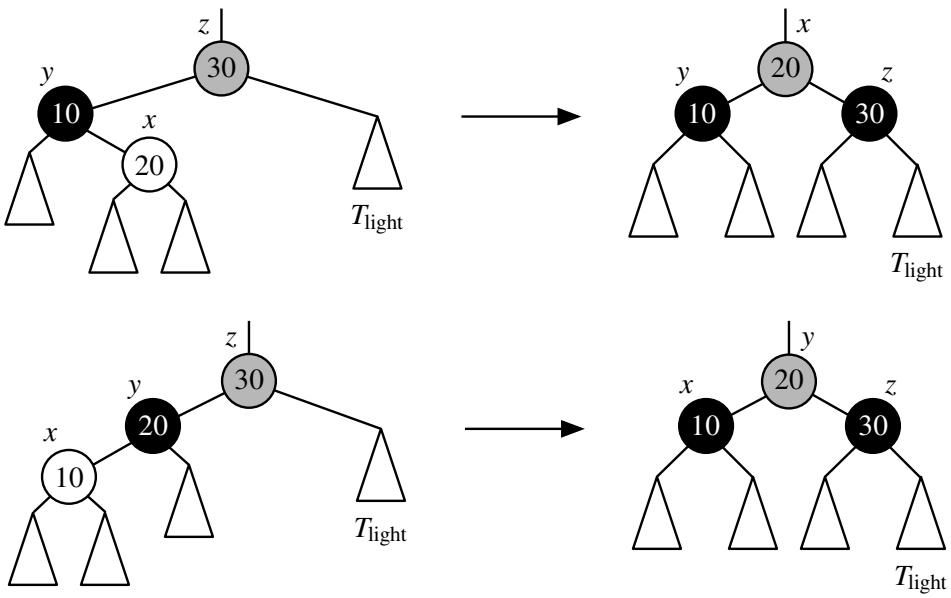


Figure 11.38: Resolving a black deficit in T_{light} by performing a trinode restructuring as restructure(x). Two possible configurations are shown (two other configurations are symmetric). The gray color of z in the left figures denotes the fact that this node may be colored either red or black. The root of the restructured portion is given that same color, while the children of that node are both colored black in the result.

Case 2: ***Node y Is Black and Both Children of y Are Black (or None).***

Resolving this case corresponds to a fusion operation in the corresponding $(2,4)$ tree T', as y must represent a 2-node. We do a ***recoloring***; we color y red, and, if z is red, we color it black. (See Figure 11.39). This does not introduce any red violation, because y does not have a red child.

In the case that z was originally red, and thus the parent in the corresponding $(2,4)$ tree is a 3-node or 4-node, this recoloring resolves the deficit. (See Figure 11.39a.) The path leading to T_{light} includes one additional black node in the result, while the recoloring did not affect the number of black nodes on the path to the subtrees of T_{heavy}.

In the case that z was originally black, and thus the parent in the corresponding $(2,4)$ tree is a 2-node, the recoloring has not increased the number of black nodes on the path to T_{light}; in fact, it has *reduced* the number of black nodes on the path to T_{heavy}. (See Figure 11.39b.) After this step, the two children of z will have the same black height. However, the entire tree rooted at z has become deficient, thereby propogating the problem higher in the tree; we must repeat consideration of all three cases at the parent of z as a remedy.

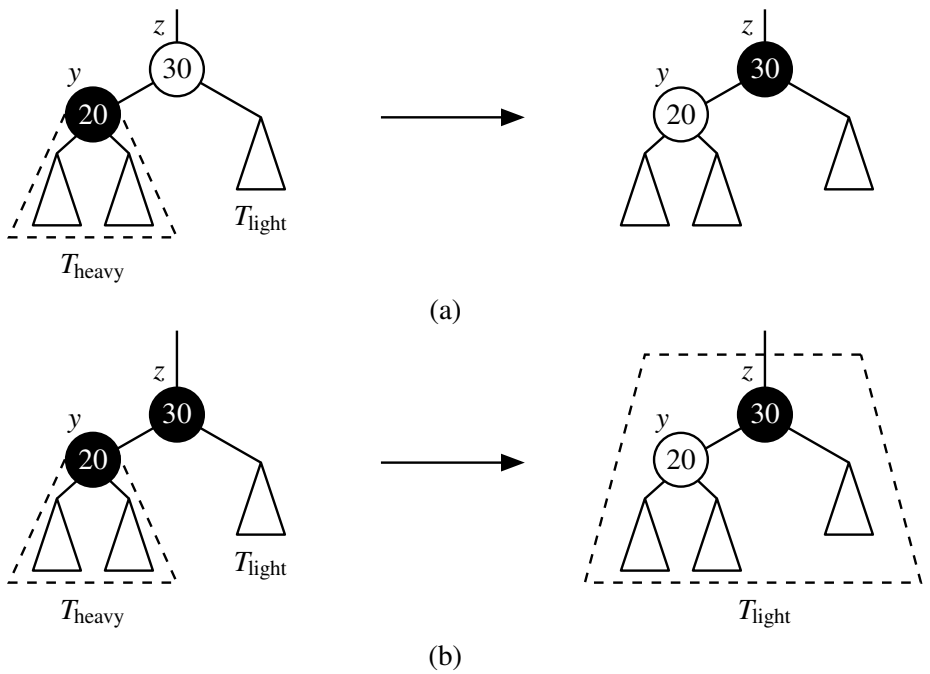


Figure 11.39: Resolving a black deficit in T_{light} by a recoloring operation: (a) when z is originally red, reversing the colors of y and z resolves the black deficit in T_{light}, ending the process; (b) when z is originally black, recoloring y causes the entire subtree of z to have a black deficit, requiring a cascading remedy.

Case 3: ***Node y Is Red***. (See Figure 11.40.)

Because y is red and T_{heavy} has black depth at least 1, z must be black and the two subtrees of y must each have a black root and a black depth equal to that of T_{heavy}. In this case, we perform a rotation about y and z, and then recolor y black and z red. This denotes a reorientation of a 3-node in the corresponding $(2,4)$ tree T'.

This does not immediately resolve the deficit, as the new subtree of z is an old subtree of y with black root y' and black height equal to that of the original T_{heavy}. We reapply the algorithm to resolve the deficit at z, knowing that the new child y', that is the root of T_{heavy} is now black, and therefore that either Case 1 applies or Case 2 applies. Furthermore, the next application will be the last, because Case 1 is always terminal and Case 2 will be terminal given that z is red.

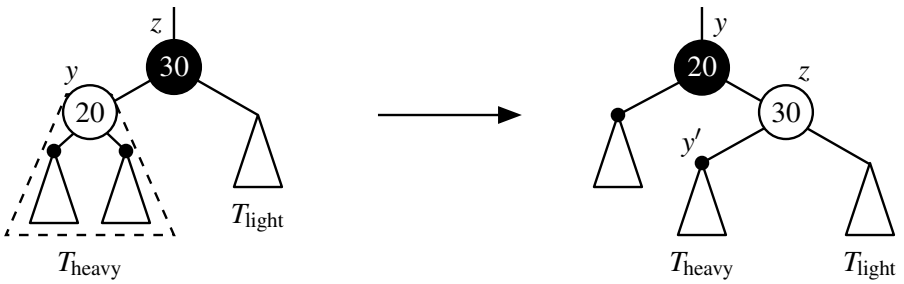


Figure 11.40: A rotation and recoloring about red node y and black node z, assuming a black deficit at z. This amounts to a change of orientation in the corresponding 3-node of a $(2,4)$ tree. This operation does not affect the black depth of any paths through this portion of the tree. Furthermore, because y was originally red, the new subtree of z must have a black root y' and must have black height equal to the original T_{heavy}. Therefore, a black deficit remains at node z after the transformation.

In Figure 11.41, we show a sequence of deletions on a red-black tree. A dashed edge in those figures, such as to the right of 7 in part (c), represents a branch with a black deficiency that has not yet been resolved. We illustrate a Case 1 restructuring in parts (c) and (d). We illustrate a Case 2 recoloring in parts (f) and (g). Finally, we show an example of a Case 3 rotation between parts (i) and (j), concluding with a Case 2 recoloring in part (k).

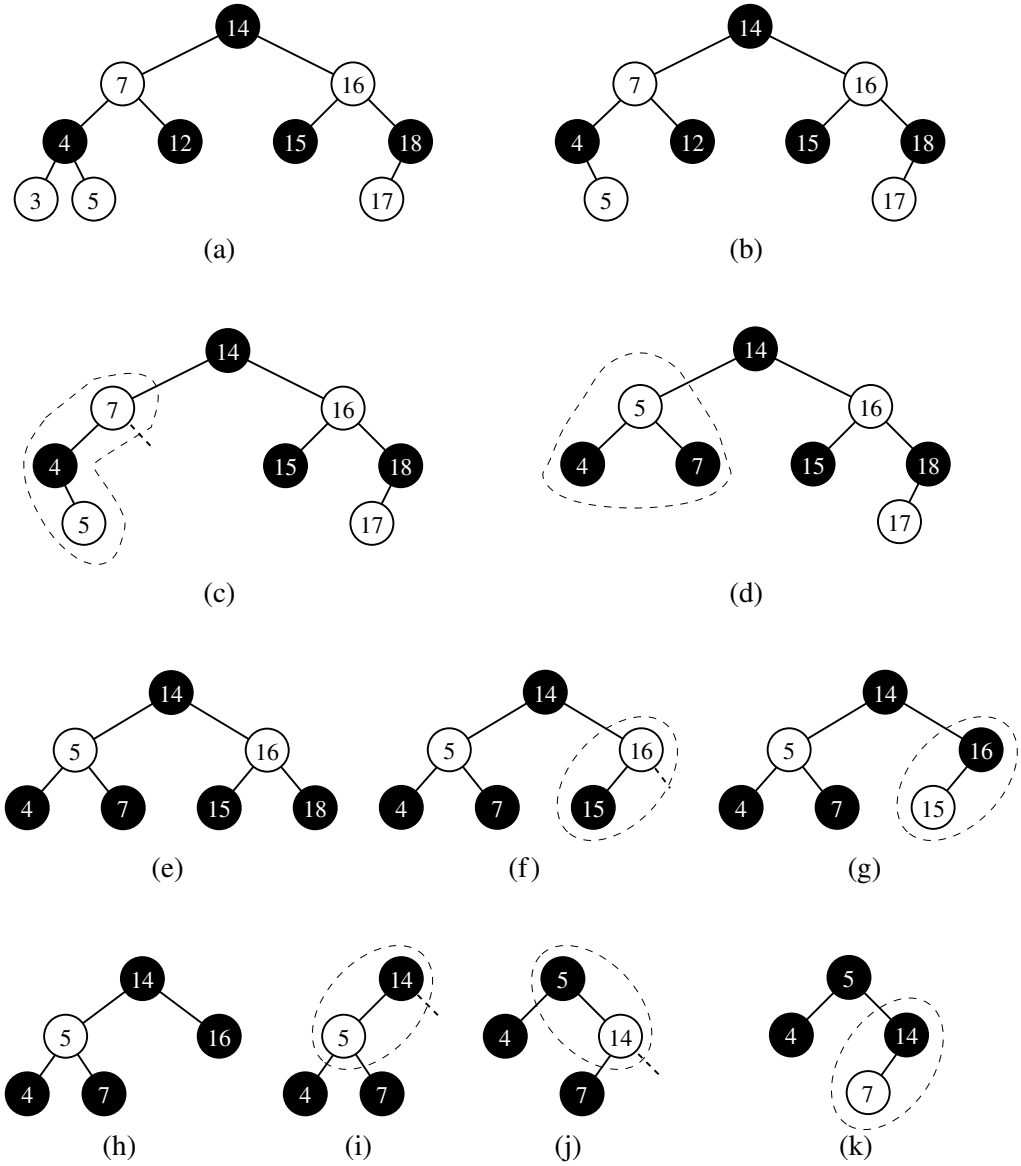


Figure 11.41: A sequence of deletions from a red-black tree: (a) initial tree; (b) removal of 3; (c) removal of 12, causing a black deficit to the right of 7 (handled by restructuring); (d) after restructuring; (e) removal of 17; (f) removal of 18, causing a black deficit to the right of 16 (handled by recoloring); (g) after recoloring; (h) removal of 15; (i) removal of 16, causing a black deficit to the right of 14 (handled initially by a rotation); (j) after the rotation the black deficit needs to be handled by a recoloring; (k) after the recoloring.

Performance of Red-Black Trees

The asymptotic performance of a red-black tree is identical to that of an AVL tree or a $(2,4)$ tree in terms of the sorted map ADT, with guaranteed logarithmic time bounds for most operations. (See Table 11.2 for a summary of the AVL performance.) The primary advantage of a red-black tree is that an insertion or deletion requires only a ***constant number of restructuring operations***. (This is in contrast to AVL trees and $(2,4)$ trees, both of which require a logarithmic number of structural changes per map operation in the worst case.) That is, an insertion or deletion in a red-black tree requires logarithmic time for a search, and may require a logarithmic number of recoloring operations that cascade upward. Yet we show, in the following propositions, that there are a constant number of rotations or restructure operations for a single map operation.

Proposition 11.10: *The insertion of an item in a red-black tree storing n items can be done in $O(\log n)$ time and requires $O(\log n)$ recolorings and at most one trinode restructuring.*

Justification: Recall that an insertion begins with a downward search, the creation of a new leaf node, and then a potential upward effort to remedy a double-red violation. There may be logarithmically many recoloring operations due to an upward cascading of Case 2 applications, but a single application of the Case 1 action eliminates the double-red problem with a trinode restructuring. Therefore, at most one restructuring operation is needed for a red-black tree insertion. ■

Proposition 11.11: *The algorithm for deleting an item from a red-black tree with n items takes $O(\log n)$ time and performs $O(\log n)$ recolorings and at most two restructuring operations.*

Justification: A deletion begins with the standard binary search tree deletion algorithm, which requires time proportional to the height of the tree; for red-black trees, that height is $O(\log n)$. The subsequent rebalancing takes place along an upward path from the parent of a deleted node.

We considered three cases to remedy a resulting black deficit. Case 1 requires a trinode restructuring operation, yet completes the process, so this case is applied at most once. Case 2 may be applied logarithmically many times, but it only involves a recoloring of up to two nodes per application. Case 3 requires a rotation, but this case can only apply once, because if the rotation does not resolve the problem, the very next action will be a terminal application of either Case 1 or Case 2.

In the worst case, there will be $O(\log n)$ recolorings from Case 2, a single rotation from Case 3, and a trinode restructuring from Case 1. ■

11.6.2 Python Implementation

A complete implementation of a RedBlackTreeMap class is provided in Code Fragments 11.15 through 11.17. It inherits from the standard TreeMap class and relies on the balancing framework described in Section 11.2.1.

We begin, in Code Fragment 11.15, by overriding the definition of the nested _Node class to introduce an additional Boolean field to denote the current color of a node. Our constructor intentionally sets the color of a new node to red to be consistent with our approach for inserting items. We define several additional utility functions, at the top of Code Fragment 11.16, that aid in setting the color of nodes and querying various conditions.

When an element has been inserted as a leaf in the tree, the _rebalance_insert hook is called, allowing us the opportunity to modify the tree. The new node is red by default, so we need only look for the special case of the new node being the root (in which case it should be colored black), or the possibility that we have a double-red violation because the new node's parent is also red. To remedy such violations, we closely follow the case analysis described in Section 11.6.1.

The rebalancing after a deletion also follows the case analysis described in Section 11.6.1. An additional challenge is that by the time the _rebalance_hook is called, the old node has already been removed from the tree. That hook is invoked on the *parent* of the removed node. Some of the case analysis depends on knowing about the properties of the removed node. Fortunately, we can reverse engineer that information by relying on the red-black tree properties. In particular, if p denotes the parent of the removed node, it must be that:

- If p has no children, the removed node was a red leaf. (Exercise R-11.26.)
- If p has one child, the removed node was a black leaf, causing a deficit, unless that one remaining child is a red leaf. (Exercise R-11.27.)
- If p has two children, the removed node was a black node with one red child, which was promoted. (Exercise R-11.28.)

```
1  class RedBlackTreeMap(TreeMap):
2    """Sorted map implementation using a red-black tree."""
3    class _Node(TreeMap._Node):
4      """Node class for red-black tree maintains bit that denotes color."""
5      __slots__ = '_red'      # add additional data member to the Node class
6
7      def __init__(self, element, parent=None, left=None, right=None):
8        super().__init__(element, parent, left, right)
9        self._red = True      # new node red by default
```

Code Fragment 11.15: Beginning of the RedBlackTreeMap class. (Continued in Code Fragment 11.16.)

```
10    #------------------------ positional-based utility methods ------------------------
11    # we consider a nonexistent child to be trivially black
12    def _set_red(self, p): p._node._red = True
13    def _set_black(self, p): p._node._red = False
14    def _set_color(self, p, make_red): p._node._red = make_red
15    def _is_red(self, p): return p is not None and p._node._red
16    def _is_red_leaf(self, p): return self._is_red(p) and self.is_leaf(p)
17
18    def _get_red_child(self, p):
19      """Return a red child of p (or None if no such child)."""
20      for child in (self.left(p), self.right(p)):
21        if self._is_red(child):
22          return child
23      return None
24
25    #------------------------ support for insertions ------------------------
26    def _rebalance_insert(self, p):
27      self._resolve_red(p)                          # new node is always red
28
29    def _resolve_red(self, p):
30      if self.is_root(p):
31        self._set_black(p)                          # make root black
32      else:
33        parent = self.parent(p)
34        if self._is_red(parent):                    # double red problem
35          uncle = self.sibling(parent)
36          if not self._is_red(uncle):               # Case 1: misshapen 4-node
37            middle = self._restructure(p)           # do trinode restructuring
38            self._set_black(middle)                 # and then fix colors
39            self._set_red(self.left(middle))
40            self._set_red(self.right(middle))
41          else:                                     # Case 2: overfull 5-node
42            grand = self.parent(parent)
43            self._set_red(grand)                    # grandparent becomes red
44            self._set_black(self.left(grand))       # its children become black
45            self._set_black(self.right(grand))
46            self._resolve_red(grand)                # recur at red grandparent
```

Code Fragment 11.16: Continuation of the RedBlackTreeMap class. (Continued from Code Fragment 11.15, and concluded in Code Fragment 11.17.)

```
47    #------------------------ support for deletions ------------------------
48    def _rebalance_delete(self, p):
49      if len(self) == 1:
50        self._set_black(self.root( ))      # special case: ensure that root is black
51      elif p is not None:
52        n = self.num_children(p)
53        if n == 1:                         # deficit exists unless child is a red leaf
54          c = next(self.children(p))
55          if not self._is_red_leaf(c):
56            self._fix_deficit(p, c)
57        elif n == 2:                       # removed black node with red child
58          if self._is_red_leaf(self.left(p)):
59            self._set_black(self.left(p))
60          else:
61            self._set_black(self.right(p))
62
63    def _fix_deficit(self, z, y):
64      """Resolve black deficit at z, where y is the root of z's heavier subtree."""
65      if not self._is_red(y): # y is black; will apply Case 1 or 2
66        x = self._get_red_child(y)
67        if x is not None: # Case 1: y is black and has red child x; do "transfer"
68          old_color = self._is_red(z)
69          middle = self._restructure(x)
70          self._set_color(middle, old_color)       # middle gets old color of z
71          self._set_black(self.left(middle))       # children become black
72          self._set_black(self.right(middle))
73        else: # Case 2: y is black, but no red children; recolor as "fusion"
74          self._set_red(y)
75          if self._is_red(z):
76            self._set_black(z)                     # this resolves the problem
77          elif not self.is_root(z):
78            self._fix_deficit(self.parent(z), self.sibling(z)) # recur upward
79      else: # Case 3: y is red; rotate misaligned 3-node and repeat
80        self._rotate(y)
81        self._set_black(y)
82        self._set_red(z)
83        if z == self.right(y):
84          self._fix_deficit(z, self.left(z))
85        else:
86          self._fix_deficit(z, self.right(z))
```

Code Fragment 11.17: Conclusion of the RedBlackTreeMap class. (Continued from Code Fragment 11.16.)